Talk overview

Harness Engineering

Every agent demo works. Then real users arrive. This is the scaffolding that keeps it standing — seven patterns, in Python, on managed Azure primitives.

Python | Microsoft Agent Framework | 64 slides · a two-hour session + Q&A

All 7 on the Agent Framework · 6 need no cloud

THE DEMO THAT DIES

Every engineering team has seen this demo. Someone wires a model to a tool or two, asks it to book a meeting or summarise a document, and the room nods: *ship it.*

Here is what the demo doesn't show you. The moment real users arrive, that agent dies four ways at once.

01 It obeys attackers.
One crafted message — *"ignore previous instructions"* — and the agent works for someone else. Nothing sits between the user and the model.

02 It loses work.
A rate limit at step nine of ten, and the run restarts from zero. Every completed step repeated, every side effect doubled.

03 It runs anything.
The model writes code, and that code executes in your process, on your filesystem, with your credentials.

04 It remembers nothing.
Every conversation starts from scratch — no history, no facts, none of the context the user gave you three turns ago.

Add two more for good measure — it answers billing questions with engineering answers, and it fires off refunds with nobody signing off — and you have the full obituary.

> **The scaffolding that prevents all of this has a name: the harness. Harness engineering — not prompt engineering — is where most of the real work of production agents lives.**

FIRST — WHICH "HARNESS"?

Not the same as OpenAI's "harness engineering." That term describes a *dev-time* practice — "humans steer, agents execute," where coding agents write your software. Great topic; different talk. This is the *runtime* meaning: the scaffolding a deployed agent runs *inside*. Same words, different layer — worth settling in the first two minutes before someone quotes it back at you.

THE TURN — PURE PYTHON, ON MICROSOFT AGENT FRAMEWORK

Build all of this by hand and you learn a great deal — and you own a great deal of infrastructure forever. Nobody writes their own scheduler any more, and the same logic applies here. So this talk is pure Python, and two layers of Microsoft's managed stack do the heavy lifting.

Azure AI Foundry — Agent Service

The *agents themselves*: model calls plus the tool loop, managed threads for memory, sandboxed built-in tools like Code Interpreter and File Search.

Microsoft Agent Framework

The *harness around them*: middleware and guardrails, durable checkpointed workflows, multi-agent orchestration, human-in-the-loop.

Our own code shrinks to a thin FastAPI layer that composes them. The working rule, written into the repo: *if a lesson can be solved by configuring a managed service, don't hand-roll it.* Seven patterns, written as orchestration rather than plumbing.

THE ONE CONVENTION THAT MADE IT WORK

There is a problem with "just use the managed service": you cannot unit-test a cloud, and you cannot develop against a subscription you are still waiting on access to. So every Azure dependency sits behind a **seam** — a `typing.Protocol` with a local implementation. The whole suite runs offline: deterministic tests, no cloud account, no API keys.

> **The second half is the half that matters: state the leak. Every stand-in names the service it replaces *and says honestly where it is weaker.***

The subprocess sandbox admits it is not a real security boundary. The keyword router admits it has no notion of meaning. The summariser admits it is lossy. This sounds like documentation hygiene; it is actually a design forcing-function — if you cannot articulate what the managed service gives you that your stub doesn't, you don't understand the pattern yet. It is also, conveniently, your migration plan.

THE SEVEN PATTERNS — ONE PER WAY THE AGENT DIED

#	THE FAILURE	THE PATTERN	MODULE

1	It obeys attackers	Agent Harness — guardrails as middleware	`guardrails.py`
2	It loses work on a crash	Durable Execution — resume from a checkpoint	`durable/`
3	It runs anything	Secure Sandboxing — isolation + hard timeout	`sandbox.py`
4	It remembers nothing	Advanced Memory — threads, recall, summarisation	`memory/`
5	It answers the wrong question	Orchestration — triage intent, hand off	`router.py`
6	One failure sinks the batch	Hierarchical Supervision — fan out, aggregate	`supervisor.py`
7	Nobody signed off	Human-in-the-Loop — pause, approve, resume	`hitl.py`

THE DEMO, LIVE — GUARDRAILS ARE AN ARMS RACE

Lesson one is twenty lines: block empty input, oversized input, and prompt-injection markers before anything reaches the model. Also trivially bypassed — swap one Latin "e" for a Cyrillic "e" and a substring check waves it straight through. So the guardrail grew a safety layer that folds homoglyphs and zero-width characters *before* matching, and a red-team corpus scores it on every test run:

```
▶ POST /chat — a prompt injection is hard-blocked
{ "guardrail": "possible prompt injection: 'ignore previous instructions'" }

▶ POST /chat — the same injection disguised with a Cyrillic 'e'
{ "guardrail": "possible prompt injection (obfuscated): 'ignore previous
instructions'" }
```

That `(obfuscated)` tag is the folding pass firing, not a lucky substring match. The guardrail is *measured*, not assumed — and the docstring still states the leak: this is heuristics, not ML. Content Safety Prompt Shields remain the production answer.

THEN WE WIRED THE REAL THING — AND IT BIT US TWICE

Reimplementing patterns locally is education; wiring them to a public-preview framework is engineering. **Preview packages drift apart.** The Azure integration package declared an open-ended dependency on a core package whose newest release had renamed an API it still imported — a fresh install broke *at import time*. The working combination turned out to be the trio uploaded to PyPI within fourteen seconds of each other. We pinned all three as a matched set and wrote the reason next to the pins, because a pin without a why is a pin someone will helpfully bump.

The second bite was subtler. The preview client drives Azure's *async* pipeline, but the obvious credential import gives you the *sync* one. Everything constructs cleanly. Every test passes. The failure appears only on the first real request — which is to say, only in production, in front of the room.

> **Two lessons in one afternoon: pin preview dependencies as matched sets, and review the code path your tests deliberately cannot reach.**

THE PAYOFF — TWO RUNTIMES, ONE SEAM

All that seam discipline buys two runtimes for the price of one interface. **Azure mode** points the Agent Framework at a Foundry project — managed threads, sandboxed tools, keyless auth. **Local mode** points the *same* framework at Foundry Local, Microsoft's on-device runtime: nothing else to install, and the first request resolves the model from the catalog, downloads it, loads it, and serves it. Phi-4-mini boots in under two minutes and answers through the full production path, entirely on a laptop.

One decision worth stealing: modes are **explicit, and explicit modes fail loud.** Ask for `azure` without credentials and the app raises with the exact variable named — it never quietly falls back to the stub. We learned that one the honest way: an earlier draft documented an opt-in flow that never actually loaded the config. Everything "worked", the stub answered politely, and only a review caught that users would believe they were testing a real model.

> **Silent fallbacks are how demos lie.**

WHAT TO TAKE AWAY

Seam pattern

A Protocol boundary with a local impl standing in for a managed service — the single swap point, and the reason your suite runs offline.

State the leak

Each stand-in documents exactly where it's weaker than the real service. No false confidence — and it doubles as the migration plan.

Thin over managed

If a lesson can be solved by configuring a managed primitive, don't build it from scratch. Own the composition, not the infrastructure.

The industry conversation is still mostly about models. The gap between an agent demo and an agent in production isn't the model — it's the harness. Guard, persist, sandbox, remember, route, supervise, ask a human. Everything else is composition.

GO DEEPER

- `docs/architecture.md` — design, SLOs, ADRs
- `docs/glossary.md` · `docs/langgraph-glossary.md`

- `docs/azure-implementation-plan.md` — the blueprint
- Run it: `make demo` · `uv run --extra dev pytest`

Companion to the 19-slide **Harness Engineering** deck (`docs/presentation/`). Local-first practice project — all seven lessons implemented and tested; Lesson 1 wired to Microsoft Agent Framework in both Azure and on-device modes.